

# Reviewer Pack — Minimal p-Adic Order of Lattices and Frege Proof Length

Entry 687c1080c597 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 11:02:09 UTC

## Minimal p-Adic Order of Lattices and Frege Proof Length

**Entry ID:** 687c1080c597 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 11:02:09 UTC

### 1. Conjecture

**Field A** (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

#### Statement:

For every Boolean satisfiability instance  $\varphi$ , the minimal order of a lattice associated with its incidence matrix is linearly correlated with the length of its Frege proof, such that  $\log(p-1)^{\min\_order(Inc(\varphi))} = \Theta(\log(2^{\{w(Frege(\varphi))\}} + 1))$ , where  $Inc(\varphi)$  is the incidence matrix of  $\varphi$ ,  $w(Frege(\varphi))$  is the depth of the Frege proof for  $\varphi$ , and  $p$  is a prime number.

#### Rationale (proposer's reasoning):

Lattice theory, particularly in the context of p-adic numbers, provides a rich framework to study algebraic structures. By linking this with Frege proof complexity, we may uncover new structural insights into the hardness of satisfiability problems. The correlation between lattice order and proof length could potentially reveal a subtle interplay between arithmetic properties and logical reasoning.

**Taxonomy category:** p-adic\_analysis\_fregeprooflength (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 9dabf6100e45d780

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the Pearson's correlation coefficient between  $\log(p-1)^{\min\_order(Inc(\varphi))}$  and  $\log(2^{\{w(Frege(\varphi))\}} + 1)$  exceeds 0.7 for all  $\varphi$  with  $n \leq 40$ , across 30 seeds. It is falsified if this coefficient is below 0.5.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 0.90       | UNCERTAIN        | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** ADJACENT\_OK against 8 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - p-adic analysis AND Boolean satisfiability proof complexity - Frege proof length IN p-adic lattices - incidence matrix AND minimal order IN Frege proof

**Top relevant hits considered:** - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for  $P$ -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/2201.08874v2] Note on  $p$ -adic Local Functional Equation - [http://arxiv.org/abs/1010.0349v2] On p-adic lattices and Grassmannians - [http://arxiv.org/abs/hep-th/9410058v3] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [http://arxiv.org/abs/2005.00091v1] Derivatives and integrals: matrix order operators as an extension of the fractional calculus - [http://arxiv.org/abs/1012.0197v2] Low-Rank Matrix Approximation with Weights or Missing Data is NP-hard - [http://arxiv.org/abs/2107.05579v4] Unifying incidence and prevalence under a time-varying general branching process

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for _ in range(n):
            clause = random.sample(variables + [f'~{v}' for v in variables], 2)
            clauses.append(' | '.join(clause))
        return ' & '.join(clauses)

    def incidence_matrix(formula, n):
        matrix = [[0] * (n + n) for _ in range(n)]
        for i, clause in enumerate(formula.split(' & ')):
            literals = clause.split(' | ')
            for literal in literals:
                if literal.startswith('~'):
                    var = literal[1:]
                    j = variables.index(var)
                    matrix[i][j] = -1
```

```

        else:
            j = variables.index(literal)
            matrix[i][j + n] = 1
    return matrix

def min_order(matrix, p):
    n = len(matrix)
    for k in range(1, n):
        for l in range(k + 1, n):
            a_k = sum(matrix[i][k] * (p ** i) % (p - 1) for i in range(n)) % (p - 1)
            a_l = sum(matrix[i][l] * (p ** i) % (p - 1) for i in range(n)) % (p - 1)
            if abs(a_k - a_l) == 1:
                return k
    return n

def frege_proof_length(formula):
    # Placeholder function to simulate Frege proof length calculation
    return len(formula.split(' & ')) * len(formula.split(' | '))

p = 2
n_max = 0
instances_tested = 0
total_metric_value = Fraction(0)

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        formula = generate_formula(n)
        matrix = incidence_matrix(formula, n)
        min_order_val = min_order(matrix, p)
        proof_length = frege_proof_length(formula)

        if min_order_val == 0 or proof_length == 0:
            continue

        metric_value = math.log(p - 1) ** min_order_val / (math.log(2 ** proof_length) + 1)
        total_metric_value += Fraction(metric_value).limit_denominator()
        instances_tested += 1
        n_max = max(n_max, n)

if instances_tested < 30:
    return {
        "metric_name": "log(p-1)^min_order(Inc(φ)) / (log(2^{w(Frege(φ))}) + 1)",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_metric_value = total_metric_value / instances_tested
return {
    "metric_name": "log(p-1)^min_order(Inc(φ)) / (log(2^{w(Frege(φ))}) + 1)",
    "metric_value": float(mean_metric_value),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": False,

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    supported_count = sum(1 for r in results if r["conjecture_holds"])
    support_fraction = Fraction(supported_count, len(results))

    if support_fraction >= Fraction(80, 100):
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} stdev={sum((r['metric_value'] - support_fraction * len(results)) ** 2 for r in results) / len(results)}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not_supported\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63746b23.py", line 103, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63746b23.py", line 65, in run_trial
    matrix = incidence_matrix(formula, n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63746b23.py", line 36, in incidence_matrix
    j = variables.index(var)
        ~~~~~^
NameError: name 'variables' is not defined

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating the Pearson’s correlation coefficient and thus confirming or refuting the conjecture. | next: Investigate the cause of the crash in the test code and run the test again to obtain results for further analysis.

## 11. Audit log (LLM calls)

**Total LLM calls:** 11

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13741        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12047        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13937        |
| 4  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9351         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8269         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9833         |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20773        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12403        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12332        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13475        |
| 11 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 14476        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140636 ms total latency. Provider mix: {'ollama\_remote': 11}

(full prompt+response transcripts available in *research/audit/687c1080c597.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/687c1080c597.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/687c1080c597.tar.gz* (if generated)